

Discusion: Analisis probabilistico y algoritmos

Integrantes

- Christian Echeverria 221441
- Gustavo Cruz 22779
- Josue Say 22801
- Mathew Cordero 22982
- Pedro Guzman 22111

1: Supongamos que se ejecutan n operaciones (entre push, pop y multipop) sobre una pila con n elementos. ¿Que operacion individual sería la mas costosa que podemos re-alizar? ¿Que tasa de crecimiento veríamos sobre el tiempo de ejecucion, si las n operaciones fueran la mas costosa?

Respuesta

La mas costosa que podemos realizar es la de multipop. Pop y push solo tienen un costo de $O(1)$ como se menciono anteriormente en el documento. En cambio multipop se hace por cada k elementos superiores a la fila o todos si es menos que k . Por ende su costo es todos los elementos de la fila o $O(n)$.

Si las n operaciones fueran las mas costosas veriamos que la tasa de crecimiento seria de $O(n^2)$. Porque en un escenario normal despues del primer multipop, en el segundo y tercero tendriamos que la pila esta vacia. Pero en este escenario la pila se rellena una y otra vez. Por lo que tendriamos n iteraciones de multipop osea $n * O(n) = O(n^2)$

2: ¿Que contradicciones encontramos en este escenario? ¿Que contradicciones encontramos si suponemos que las n operaciones se efectuan sobre una pila inicialmente vacia? Hint: ya lo mencionamos hace algunos slides.

Solucion

El problema es que esto no se mantiene en el tiempo. Ya que al terminar la iteracion primera del multipop ya no hay manera de tener mas elementos de nuestra pila.

De hecho si inicialmente nuestra pila esta vacia aqui vemos que mencionaron anteriormente. “El número de iteraciones del ciclo while es el mínimo de s y k objetos sacados de la pila.”.

Esto quiere decir que no podemos realizar mas operaciones pop o multipop que operaciones push, porque es necesario algo en la lista para poder sacarlo. Esto sigue con la misma contradiccion que el numero de elementos extraidos no excede el numero de elementos insertados.

Por ende el costo seria de $O(1)$ y no $O(n)$.

3: ¿Cual es el tiempo de ejecucion de una secuencia realista de n operaciones push, pop y/o multipop sobre una pila inicialmente vacia? ¿Como se afectan las operaciones push y pop entre si, al respecto de su proporcion en las n operaciones?

Solucion

La secuencia realista entonces seria de $O(n)$. Que es el tiempo total de elementos extraidos por el multipop, y de $O(1)$ por cada operacion de pop y push.

Sabemos que - Cada elemento puede ser insertado en la pila sólo una vez (mediante push)

- Cada elemento puede ser eliminado de la pila sólo una vez (mediante pop o como parte de multipop)

Por ende el unico tiempo sera el afectado por $O(n)$ de las operaciones multipop de la pila.

Las operaciones se verian afectadas entre si segun su proporcion debido a que la pila aunque crezca el tiempo es $O(n)$. Y aunque existan mas pop no podran sacar mas. La proporción entre push y pop afecta al tamaño máximo de la pila durante la ejecución, lo que impacta en el uso de memoria, pero sigue siendo el mismo tiempo de ejecucion independientemente.

4 En ‘promedio’, ¿cuánto contribuye cada operación de la secuencia a este tiempo de ejecución? ¿Qué diferencia hay entre este costo amortizado y el costo real de cada operación?

En promedio, cada operación contribuye con un costo amortizado de $O(1)$ al tiempo de ejecución total. Esto significa que aunque algunas operaciones indi-

viduales puedan ser costosas (como MULTIPOP que podría tener un costo de $O(n)$ en el peor caso), al considerar una secuencia de n operaciones, el costo total es $O(n)$, dando un costo promedio constante por operación. La diferencia entre este costo amortizado y el costo real de cada operación es que:

El costo real puede variar significativamente entre operaciones. Algunas operaciones como POP o PUSH tienen costo constante de 1, mientras que MULTIPOP puede tener un costo de hasta n (el tamaño de la pila). El costo amortizado distribuye el costo total entre todas las operaciones, ofreciendo una “media” que permite analizar mejor el rendimiento en el tiempo. Las operaciones costosas (como MULTIPOP) son “compensadas” por operaciones más baratas (como PUSH o POP), resultando en un costo amortizado constante para la secuencia completa.

5 ¿De qué, principalmente, depende el tiempo de ejecución de este algoritmo? Tomando esto en cuenta, ¿cuál sería el worst-case scenario y cuál sería la cota pesimista para el tiempo de ejecución de n operaciones INCREMENT?

El tiempo de ejecución del algoritmo INCREMENT depende principalmente de la cantidad de bits que deben modificarse (bits que cambian de 1 a 0 o de 0 a 1) en cada operación. Específicamente, cuando realizamos un incremento, el número de bits modificados puede variar. En el mejor caso, solo se modifica un bit (cambiar un 0 a 1). En el peor caso, se modifican todos los bits del contador (cuando todos los bits son 1 y deben cambiar a 0, y se agrega un 1 adicional). El worst-case scenario ocurre cuando tenemos que realizar un incremento que requiere modificar todos los k bits del contador. Esto sucede cuando el contador tiene todos sus bits en 1 (por ejemplo, cuando es

$$2^k - 1$$

y se incrementa a

$$2^k$$

). La cota pesimista para el tiempo de ejecución de

$$n$$

operaciones INCREMENT sería

$$O(nk)$$

si consideramos el peor caso para cada operación. Sin embargo, el análisis amortizado demuestra que el costo total es en realidad

$$O(n)$$

, lo que significa que el costo amortizado por operación es

$$O(1)$$

, independientemente del número de bits que se modifiquen en una operación particular. Esto es posible porque las operaciones costosas (que modifican muchos bits) ocurren con poca frecuencia, y el análisis amortizado distribuye este costo entre todas las operaciones.

Pregunta 6

En general, ¿cuántas veces cambiará el i -ésimo bit (de derecha a izquierda) al hacer n incrementos a un número binario con k bits? ¿Qué proporción de n acotaría por arriba el costo total de los n incrementos al número? Investigue la serie geométrica y presente su procedimiento para encontrar la cota.

1. ¿Cuántas veces cambia el i -ésimo bit?

- Cada vez que se incrementa el contador, el bit en la posición i cambia de 0 a 1 solo si los bits de menor peso han generado un acarreo que lo afecta.
- Esto sucede cada 2^i incrementos.

Entonces, en n incrementos, el i -ésimo bit cambiará:

$$\left\lfloor \frac{n}{2^i} \right\rfloor \text{ veces}$$

2. ¿Cuál es el costo total de los cambios de bits?

El costo total es la suma de todos los cambios de bits:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq n \sum_{i=0}^{\infty} \frac{1}{2^i}$$

La serie geométrica converge a:

$$\sum_{i=0}^{\infty} \frac{1}{2^i} = \frac{1}{1 - \frac{1}{2}} = 2$$

Por lo tanto, el costo total de los n incrementos es:

$$\mathcal{O}(n) \times 2 = \mathcal{O}(n)$$

3. ¿Qué proporción de n acota por arriba el costo total?

La proporción es $2n$, por la convergencia de la serie. Esto nos permite afirmar que el costo total de los cambios de bits en n incrementos es a lo sumo $2n$, es decir, sigue siendo lineal.

Pregunta 7

¿Qué diferencias podemos percibir respecto a los costos que asigna este método, comparado con el método de agregación? ¿Qué debemos asegurar al asignar los costos de cada operación para que la secuencia se complete con éxito? ¿Durante la ejecución de la secuencia, podríamos estar algún momento “en deuda”?

1. Diferencias entre el Método de Agregación y el Método Contable

- **Método de Agregación:** Suma el costo total de las operaciones y divide entre n . Se obtiene un único costo promedio para todas las operaciones.
- **Método Contable:** Permite asignar costos artificiales (mayores o menores) a las operaciones. Esto ayuda a manejar mejor las operaciones que anticipan costos futuros (por ejemplo, `push` paga por su propio costo y por un futuro `pop`).

2. ¿Qué debemos asegurar al asignar los costos de cada operación?

- Que el crédito nunca sea negativo.
- Formalmente, se debe cumplir la desigualdad:

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

Esto garantiza que los créditos acumulados cubren los costos futuros, evitando un “déficit” en la estructura de datos.

3. ¿Durante la ejecución podríamos estar ‘en deuda’?

No. Si el crédito es negativo, significa que hemos subestimado el costo de operaciones pasadas, y no podremos cubrir los costos reales futuros. Esto rompe la validez del análisis amortizado.

Pregunta 8

Si hemos insertado n elementos, ¿cuántas expansiones (potencias de dos) habrán ocurrido? Complete la desigualdad que acota el tiempo de ejecución de las n inserciones:

$$\sum_{i=1}^n c_i \leq n + \dots$$

1. ¿Cuántas expansiones han ocurrido al insertar n elementos?

- Cada vez que la tabla se llena, se duplica su capacidad.
- Las expansiones ocurren en las potencias de dos:
 - Primera expansión cuando insertamos el 1er elemento (capacidad 1).
 - Segunda expansión al insertar el 2do elemento (capacidad 2).
 - Tercera expansión al insertar el 4to elemento (capacidad 4).
 - Y así sucesivamente.
- En total, se han hecho:

$$\lfloor \log_2 n \rfloor \text{ expansiones.}$$

2. Costo de las Inserciones

- Cada inserción tiene un costo de 1, excepto cuando hay expansión, en cuyo caso también pagamos por copiar los elementos existentes.
- El costo total es:

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j$$

- La segunda suma es el costo de copiar los elementos durante cada expansión:

$$\sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j = 2^{\lfloor \log_2 n \rfloor + 1} - 1 \leq 2n - 1$$

3. Desarrollo de la Desigualdad

$$\sum_{i=1}^n c_i \leq n + (2n - 1) = 3n - 1 \in \mathcal{O}(n)$$

9: ¿Cuál es la fórmula para calcular el costo amortizado de una operación con esta perspectiva? ¿Cuál sería la fórmula para el costo amortizado total de una secuencia de n operaciones?

Método del Costo Agregado

Se toma el costo real de cada operación y se “promedia” sobre todas las operaciones de la secuencia.

Formula:

- Sea c_i el costo real de la operación i .
- El **costo total real** de n operaciones es

$$C_{\text{total}} = \sum_{i=1}^n c_i.$$

- El **costo amortizado por operación** (constante) es

$$\hat{c} = \frac{C_{\text{total}}}{n} = \frac{1}{n} \sum_{i=1}^n c_i.$$

3. Fórmula final

$$\hat{c} = \frac{1}{n} \sum_{i=1}^n c_i.$$

4. Costo amortizado total de la secuencia de n operaciones:

$$n \times \hat{c} = n \times \frac{1}{n} \sum_{i=1}^n c_i = \sum_{i=1}^n c_i = C_{\text{total}}.$$

2. Método de la Función de Potencial

Asociamos a cada estado del dato (estructura) una “energía” o potencial Φ . Si el estado pasa de D_{i-1} a D_i tras la operación i , el cambio de potencial amortigua picos de costo.

Definiciones:

- c_i : costo real de la operación i .
- $\Phi(D)$: potencial asociado al estado D .

- Se requiere $\Phi(D) \geq 0$ y típicamente $\Phi(D_0) = 0$ (estado inicial).

Costo amortizado de la operación i :

$$\hat{c}_i = c_i + [\Phi(D_i) - \Phi(D_{i-1})].$$

Desglose:

- c_i : gastas “realmente” esta cantidad.
- $\Phi(D_i) - \Phi(D_{i-1})$:
 - Si sube el potencial (+), “guardas” crédito.
 - Si baja (−), usas crédito guardado.

Costo amortizado total de las n operaciones:

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0).$$

Si $\Phi(D_0) = 0$,

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n).$$

Fórmulas finales

- **Costo amortizado por operación (agregado):**

$$\hat{c} = \frac{1}{n} \sum_{i=1}^n c_i.$$

- **Costo amortizado total (agregado):**

$$n \cdot \hat{c} = \sum_{i=1}^n c_i.$$

- **Costo amortizado de la operación i (potencial):**

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

- Costo amortizado total (potencial):

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0).$$

10. ¿Qué escenario problemático podría enfrentar nuestra fórmula para el potencial bi? ¿Qué sí podemos asegurar acerca de bi en relación con bi−1 −ti +1 tomando en cuenta este escenario? ¿Cómo ajustamos la fórmula de potencial para tomar en cuenta el escenario problemático?

Escenario problemático

Caso wrap-around: cuando $b_i = 0$, la operación de incremento ha restablecido todos los k bits a 0 (es decir, $t_i = k$ y $b_{i-1} = k$). En ese momento, no se cumple la igualdad

$$b_i = b_{i-1} - t_i + 1$$

porque el contador vuelve a cero en lugar de quedar en 1.

Aunque la igualdad falle, sí es cierto que en todos los casos

$$\boxed{b_i \leq b_{i-1} - t_i + 1.}$$

- Cuando no hay wrap-around ($b_i > 0$), de hecho se cumple igualdad.
- Cuando hay wrap-around ($b_i = 0$), entonces

$$b_{i-1} = k, \quad t_i = k \quad \implies \quad b_{i-1} - t_i + 1 = 1 \quad \implies \quad 0 = b_i \leq 1.$$

Ajuste a la diferencia de potencial

Partíamos de

$$\Delta\Phi_i = \Phi(D_i) - \Phi(D_{i-1}) = b_i - b_{i-1}.$$

Usando $b_i \leq b_{i-1} - t_i + 1$ obtenemos

$$\Delta\Phi_i \leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i.$$

Nuevo cálculo del costo amortizado

El costo real de la operación i es

$$c_i \leq t_i + 1$$

(porque se resetean t_i bits y se pone a 1 un bit). Por tanto, el costo amortizado queda:

$$\begin{aligned} a_i &= c_i + \Delta\Phi_i \leq (t_i + 1) + (1 - t_i) \\ &= 2. \end{aligned}$$

Y de aquí se concluye que, para cualquier secuencia de n incrementos,

$$\sum_{i=1}^n a_i = O(n) \implies \sum_{i=1}^n c_i = O(n).$$

11. Que cambio a las propiedades de la pila provocan las operaciones push, pop y multipop? Proponga una funcion de potencial para este problema. Luego provea el costo amortizado para la operacion push usando la funcion de potencial propuesta.

Para esto vamos a ver que push aumenta el numero de elementos en la pila en 1, pop los disminuye en 1 y multipop hasta k.

Para la funcion de potencial proponemos la siguiente

$$\Phi(D) = c * |S|$$

Donde $|S|$ es el numero de elementos de nuestra pila que es S.

Para calcular push usamos el metodo del metodo del potencial.

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$$

Para la operacion push sabemos que es lo siguiente

- $\hat{c}_i = 1$ (costo amortizado de la operacion i que es siempre constante)
- $\Phi_{i-1} = |S|$ (potencial antes del push que es la cantidad que tenemos en nuestra pila ya que no hemos modificado nada)
- $\Phi_i = |S| + 1$ (potencial despues del push que nuestro pila aumenta 1 en tamaño)

Al final tendríamos que

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$$

$$\hat{c}_i = 1 + (|S| + 1) - (|S|)$$

$$\hat{c}_i = 1 + |S| + 1 - |S|$$

$$\hat{c}_i = 1 + 1$$

$$\hat{c}_i = 2$$

Como vemos el costo es de 2 en operacion push, lo que significa que es constante o sea push es

$$O(1)$$

12 Análisis Amortizado de Operaciones de Pila

Función Potencial

- **Definición:**
 $\Phi(D_i)$ = número de elementos en la pila.
- **Pila vacía inicial:**
 $\Phi(D_0) = 0$.
- **Propiedad clave:**
 $\Phi(D_i) \geq 0$ para toda i (el tamaño de la pila nunca es negativo). — ##
Cálculo de Costos Amortizados ### 1. Operación POP
- **Costo real (c_i):**
1 (eliminar 1 elemento).

- **Cambio en el potencial ($\Delta\Phi$):**
 $\Phi(D_i) - \Phi(D_{i-1}) = (\text{tamaño}(D_{i-1}) - 1) - \text{tamaño}(D_{i-1}) = -1.$
- **Costo amortizado (c'_i):**
 $c'_i = c_i + \Delta\Phi = 1 + (-1) = 0.$

Justificación:

El costo real de POP se compensa con la disminución del potencial. El potencial acumulado por operaciones PUSH anteriores cubre este costo. — ### 2.

Operación MULTIPOP(k) - **Costo real** (c_i):

$$j = \min(k, \text{tamaño}(D_{i-1})) \text{ (eliminar } j \text{ elementos).}$$

- Cambio en el potencial ($\Delta\Phi$):

$$\Phi(D_i) - \Phi(D_{i-1}) = (\text{tamaño}(D_{i-1}) - j) - \text{tamaño}(D_{i-1}) = -j.$$

- **Costo amortizado (c'_i):**
 $c'_i = c_i + \Delta\Phi = j + (-j) = 0.$

Justificación:

Cada elemento eliminado reduce el potencial en 1, igualando el costo real. El potencial acumulado por PUSH absorbe el gasto. — ## Análisis Asintótico

de una Secuencia de n Operaciones ### Costos Amortizados por Operación |

Operación	Costo Real	Cambio de Potencial	Costo Amortizado	—
-----------	------------	---------------------	------------------	---

—————|—————|—————|—————| | PUSH | 1 | +1 | 2

POP	1	-1	0	MULTIPOP(k)	$j \leq k$	-j	0	### Costo Total Amorti-
-----	---	----	---	-------------	------------	----	---	-------------------------

$$\text{zado } T_{\text{amortizado}} = (\text{número de PUSH}) \times 2 + (\text{número de POP y MULTIPOP}) \times$$

0.

Dado que el número de operaciones PUSH no excede n :

$$T_{\text{amortizado}} \leq 2n. \quad \text{Relación con el Costo Real } T_{\text{real}} = T_{\text{amortizado}} + \Phi(D_0) - \Phi(D_n).$$

Como $\Phi(D_n) \geq 0$ y $\Phi(D_0) = 0$:

$$T_{\text{real}} \leq T_{\text{amortizado}} \leq 2n. \text{ — } \#\# \text{ Tasa de Crecimiento Asintótico - Conclusión:}$$

Cualquier secuencia de n operaciones tiene un **costo total real** de $O(n)$.

- Notación Asintótica:

$O(n)$

Explicación Clave:

- Los costos altos de MULTIPOP se “pagan” con el potencial acumulado por PUSH.
- Cada operación contribuye en promedio con $O(1)$ al costo total.
- La complejidad total crece linealmente con n .